

Rapport de Débogage et Refactoring Architectural

Suppression des `Optional` et Résolution des Récursions Infinies (Projet JGrapa)

Charles Baudoux

8 juillet 2026

Résumé

Ce rapport détaille les étapes de refactoring effectuées sur le module d'agrégation du projet JGrapa. L'objectif principal était de simplifier l'architecture en remplaçant l'utilisation des `Optional` par des références directes pouvant être nulles, tout en résolvant les erreurs en cascade (`NullPointerException`, `StackOverflowError`) induites par ce changement structurel.

Table des matières

1	Introduction	1
2	Nettoyage des Constructeurs et de l'Héritage	2
2.1	Avant la modification	2
2.2	Après la modification	2
3	Retrait des méthodes associées à <code>Optional</code>	2
3.1	Avant la modification	2
3.2	Après la modification	3
4	Résolution du <code>StackOverflowError</code> (Boucle Infinie)	3
4.1	Avant la modification	3
4.2	Après la modification	3
5	Adaptation du Convertisseur JSON et des Tests	4
5.1	Avant la modification (Tests Schema)	4
5.2	Après la modification	4
6	Conclusion	4

1 Introduction

L'architecture initiale du système d'agrégation (classes `Aggregator`, `Weighter`, `Parametric`, `Owa`) utilisait `Optional<Aggregator>` pour représenter l'absence de sous-agrégateur par défaut (`defaultSub`). Cette approche a été jugée trop lourde. Le refactoring a consisté à remplacer ces boîtes par des objets directs (acceptant `null`). Ce changement a nécessité une mise à jour en profondeur des constructeurs, des méthodes de conversion JSON, et a mis en lumière un problème architectural de boucle infinie.

2 Nettoyage des Constructeurs et de l'Héritage

La première étape a été de modifier la signature de la classe parente `Aggregator` et de toutes ses classes filles pour qu'elles acceptent directement un `Aggregator` au lieu d'un `Optional`.

2.1 Avant la modification

L'utilisation de `Optional` forçait l'emballage systématique des paramètres et l'utilisation de `checkNotNull` sur une valeur qui était censée représenter une absence.

```
1 protected Aggregator(Map<Criterion, Aggregator> subs, Optional<Aggregator>
   defaultSub) {
2     this.subs = ImmutableMap.copyOf(subs);
3     this.defaultSub = checkNotNull(defaultSub);
4 }
5
6 // Appel dans la classe fille (ex: Owa)
7 super(subs, defaultSub == Weighter.FULL_EQUAL_WEIGHTER ? Optional.empty() :
   Optional.of(defaultSub));
```

Listing 1 – Constructeur initial avec `Optional`

2.2 Après la modification

La classe accepte désormais la valeur `null` de manière native et légitime.

```
1 protected Aggregator(Map<Criterion, Aggregator> subs, Aggregator defaultSub)
   {
2     this.subs = ImmutableMap.copyOf(subs);
3     this.defaultSub = defaultSub; // checkNotNull retire car null est
   accepte
4 }
5
6 // Appel dans la classe fille (ex: Owa)
7 super(subs, defaultSub == Weighter.FULL_EQUAL_WEIGHTER ? null : defaultSub);
```

Listing 2 – Nouveau constructeur simplifié

3 Retrait des méthodes associées à `Optional`

Le changement de type a rendu obsolètes les méthodes `.ifPresent()` et `.orElse()` de l'API `Stream` de Java, causant des erreurs de compilation.

3.1 Avant la modification

```
1 public Aggregator defaultSub() {
2     return defaultSub.orElse(Weighter.FULL_EQUAL_WEIGHTER);
3 }
4
5 // Utilisation dans une boucle
6 defaultSub.ifPresent(agg -> subAggregators.put(criterion, agg));
```

Listing 3 – Logique de fallback avec `Optional`

3.2 Après la modification

Nous sommes revenus à des vérifications de nullité standard et à l'utilisation d'opérateurs ternaires.

```
1 public Aggregator defaultSub() {
2     return defaultSub != null ? defaultSub : Weighter.FULL_EQUAL_WEIGHTER;
3 }
4
5 // Utilisation dans une boucle
6 if (defaultSub != null) {
7     subAggregators.put(criterion, defaultSub);
8 }
```

Listing 4 – Remplacement par des tests de nullite

4 Résolution du StackOverflowError (Boucle Infinie)

La modification du fallback (FULL_EQUAL_WEIGHTER) a provoqué un `StackOverflowError` lors de l'exécution des tests. Ce bug se produisait dans la méthode `equals` : deux objets `Weighter` par défaut tentaient de comparer leurs `defaultSub()`, qui renvoyaient eux-mêmes, créant une récursivité infinie.

4.1 Avant la modification

La comparaison classique forçait l'appel à `equals` sur le sous-agrégateur sans condition d'arrêt si celui-ci pointait vers lui-même.

```
1 @Override
2 public boolean equals(Object o2) {
3     if (!(o2 instanceof Weighter)) return false;
4     final Weighter t2 = (Weighter) o2;
5     return weights.equals(t2.weights)
6         && subs().equals(t2.subs())
7         && defaultSub().equals(t2.defaultSub()); // Point de rupture
8 }
```

Listing 5 – Methode equals provoquant la boucle

4.2 Après la modification

L'utilisation de `Objects.equals()` permet un court-circuit de référence (comparaison d'adresses mémoires) avant de plonger dans l'arbre des objets, stoppant ainsi la récursivité.

```
1 @Override
2 public boolean equals(Object o2) {
3     if (!(o2 instanceof Weighter)) return false;
4     final Weighter t2 = (Weighter) o2;
5     return weights.equals(t2.weights)
6         && subs().equals(t2.subs())
7         && Objects.equals(defaultSub(), t2.defaultSub()); // Coupe la boucle
8 }
```

Listing 6 – Sécurisation avec `Objects.equals`

5 Adaptation du Convertisseur JSON et des Tests

La sérialisation JSON entraînait également dans une boucle infinie à cause du `defaultSub` qui s'auto-référençait. Enfin, les tests unitaires ont dû être adaptés à l'environnement d'exécution (anglais vs français).

5.1 Avant la modification (Tests Schema)

L'assertion dépendait de la langue locale du système d'exploitation, rendant le test instable.

```
1 assertTrue(e0.getMessage().contains("enumeration"), e0.getMessage());
```

Listing 7 – Test dépendant de la locale

5.2 Après la modification

Pour garantir la robustesse du test sur n'importe quel environnement (intégration continue, différents systèmes d'exploitation), la `Locale` a été fixée explicitement sur l'anglais pour la durée du test.

```
1 @Test
2 void testWrongSchema() throws Exception {
3     // Force l'environnement pour rendre le test predictable
4     java.util.Locale.setDefault(java.util.Locale.ENGLISH);
5
6     // ... code de validation ...
7
8     // Test strict sur les cles anglaises de la bibliotheque
9     assertTrue(e0.getMessage().contains("enumeration"), e0.getMessage());
10    assertTrue(e1.getMessage().contains("array expected"), e1.getMessage());
11 }
```

Listing 8 – Test robuste bilingue et securite JSON

6 Conclusion

Le refactoring s'est soldé par la suppression totale de l'overhead lié aux objets `Optional` dans l'architecture d'agrégation. L'intégrité de l'arbre a été préservée, les boucles infinies de référencement croisé (StackOverflow) ont été identifiées et neutralisées via `Objects.equals`, et les tests ont été fiabilisés. L'ensemble des 38 tests passe désormais avec succès.